Laxmi Charitable Trust's

Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce

Department of Information Technology (B.Sc.I.T Semester V)

Dot .Net Core and Progressive Web Application Practical

Practical – AI M1 PRACTICAL NO. 7 - PART 4 & 5

Roll No.: T034	Name: Rabiya
Class: TYIT	Batch: 01
Date of Assignment: 07-08-26	Date/Time of Submission: 07-08-26

4. **Implement k-fold cross-validation on a classification model.**

Code

```
from sklearn.datasets import load_iris

from sklearn.model_selection import cross_val_score

from sklearn.neighbors import KNeighborsClassifier

# Load dataset

iris = load_iris()

X = iris.data

y = iris.target

# Create classification model

model = KNeighborsClassifier()

# Apply 5-fold cross-validation

scores = cross_val_score(model, X, y, cv=5)

print("Accuracy scores for each fold:")

print(scores)

print("Average Accuracy:", round(scores.mean() * 100, 2), "%")
```

Output

```
Accuracy scores for each fold:
[0.96666667 1.         0.93333333 0.96666667 1.        ]
Average Accuracy: 97.33 %
```

NetFlix dataset

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.preprocessing import LabelEncoder
# Load dataset
df = pd.read_csv("C:/Users/info/Downloads/NetFlix - NetFlix.csv ")
# Select required columns
df = df[['type', 'release_year', 'rating']]
# Remove missing values
df = df.dropna()
# Convert text to numbers
le_type = LabelEncoder()
le_rating = LabelEncoder()
df['type'] = le_type.fit_transform(df['type'])
df['rating'] = le_rating.fit_transform(df['rating'])
# Features and Target
X = df[['release_year', 'rating']]
y = df['type']
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# Create model
model = DecisionTreeClassifier(random_state=42)
# Train model
model.fit(X_train, y_train)
# Accuracy
print("Training Accuracy:", model.score(X_train, y_train) * 100)
print("Testing Accuracy:", model.score(X_test, y_test) * 100)
# Prediction
prediction = model.predict([[2020, 3]])
print("Predicted Type:", le_type.inverse_transform(prediction))
```

5. Implement a Decision Tree classifier using the Iris or any relevant dataset.

```
Training Accuracy: 73.00771208226222
Testing Accuracy: 71.72236503856041
Predicted Type: ['Movie']
```

output

5. Implement a Decision Tree classifier using the Iris or any relevant dataset.

Code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
# Load dataset
iris = load_iris()
X = iris.data
y = iris.target
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)
# Create model
model = DecisionTreeClassifier()
# Train model
model.fit(X_train, y_train)
# Display accuracy
print("Training Accuracy:", model.score(X_train, y_train) * 100)
print("Testing Accuracy:", model.score(X_test, y_test) * 100)
# Prediction
prediction = model.predict([X_test[0]])
print("Predicted Class:", prediction[0])
```

```
Training Accuracy: 100.0
Testing Accuracy: 95.55555555555556
Predicted Class: 0
```

NetFlix dataset

Code

```
import pandas as pd
from sklearn.model_selection import cross_val_score
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import LabelEncoder
# Load dataset
df = pd.read_csv("C:/Users/info/Downloads/NetFlix - NetFlix.csv ")
# Select required columns
df = df[['type', 'release_year', 'rating']]
# Remove missing values
df = df.dropna()
# Encode categorical columns
le_type = LabelEncoder()
le_rating = LabelEncoder()
df['type'] = le_type.fit_transform(df['type'])
df['rating'] = le_rating.fit_transform(df['rating'])
# Features and Target
X = df[['release_year', 'rating']]
y = df['type']
# Create model
model = RandomForestClassifier(n_estimators=100, random_state=42)
# Apply 5-Fold Cross Validation
scores = cross_val_score(model, X, y, cv=5)
print("Accuracy Scores:")
print(scores)
print("Average Accuracy:", round(scores.mean() * 100, 2), "%")
```

Output

```
Accuracy Scores:
[0.70694087 0.70822622 0.71208226 0.72172237 0.71529563]
Average Accuracy: 71.29 %
```